Awesome 😄 Let’s switch patterns now.

Before, we did this:

Python

```
steps = [4500, 8200, 10000]

for i in range(len(steps)):
    print(i, steps[i])
```

Now, Python can hand you each value directly, so you don’t even touch indices:

Python

```
steps = [4500, 8200, 10000]

for value in steps:
    print(value)
```

Here’s what’s happening, step by step:

for value in steps: means “go through steps from start to end”
Each loop, value becomes the next item: first 4500, then 8200, then 10000
print(value) just prints that current item

So you still visit everything, but the code is shorter and cleaner when you don’t need positions.

Does this “**直接** over the values” pattern feel clear so far, or should I walk through one loop cycle in slow motion?

It's clear so far

What do you think? Pick an answer in the question box below 👇

Given this list:

```
names = ["Ron", "Alex", "Jordan"]

for name in names:
    print(name)
```

How many lines will be printed, and what will the first line be?

A.

2 lines, first line: Ron

B.

3 lines, first line: Ron

C.

3 lines, first line: Alex

D.

4 lines, first line: Ron

Nice, Ron — you got it again 🎯

You were right:

The list has 3 names → the loop runs 3 times → 3 lines
In a for name in names loop, Python starts at the first element "Ron"
So the first line printed is Ron

So “3 lines, first line: Ron” is exactly correct ☑

Now let’s connect both patterns with a tiny, real task.

Imagine:

Python

```
scores = [80, 95, 72, 88]
```

Two ways we could loop:

By index (when we care about positions):
for i in range(len(scores)): → use scores[i]
By value (when positions don’t matter):
for score in scores: → just use score
Next, I’d like you to practice both patterns in the editor:

A loop that uses indices
A loop that uses values

Are you up for a small coding task for each pattern?

Yes, I'm ready

Great 👍 Let’s do two small tasks, starting with the index pattern.

In the code editor (not in chat), please write code that:

Uses this list:

Python

```
numbers = [5, 10, 15, 20, 25, 30]
```

Uses a for loop with range(len(numbers))

Prints every other item’s index and value (so indices 0, 2, 4)

Hint structure (don’t copy, just use the idea):

Python

```
for i in range( ... ):
    print(i, numbers[i])
```

But adjust range(...) so it skips every other index.

Write and Run your code in the editor.
When it works, hit Submit code so I can see it.
I’ll check it, then we’ll do the second pattern (looping directly over values).
Python

```
numbers = [5, 10, 15, 20, 25, 30]
for i in range(0,len(numbers),2):
    print(i, numbers[i])
```

Nice job, Ron — that’s a clean index-based loop 💯

You used:

Python

```
for i in range(0, len(numbers), 2):
    print(i, numbers[i])
```

0 → start at index 0
len(numbers) → stop before the length

2 → step of 2, so you hit indices 0, 2, 4
And the output matches exactly: 0 5, 2 15, 4 25 ☑
Now let’s practice the value-based pattern.

In the code editor, write a new program that:

Uses the same list:

numbers = [5, 10, 15, 20, 25, 30]
Loops directly over the values (no range, no indices)

Computes the average of all the numbers

At the end, print the average

Structure hint:

Python

```
numbers = [5, 10, 15, 20, 25, 30]

# maybe start with total = 0
# loop: for num in numbers:
# after the loop, divide by len(numbers)
# print the result
```

Write and Run it in the editor.
When it looks good, hit Submit code and I’ll review it.
Python

```
total = 0
numbers = [5, 10, 15, 20, 25, 30]
for number in numbers:
    total = total + number
average = total / len(numbers)
print(f"The average of {numbers} is {average}")
```

Ron, this is excellent 🤝

You used the value-based loop perfectly:

Python

```
for number in numbers:
    total = total + number
```

Step by step, your code does:

Starts with total = 0
Adds each value in numbers to total
After the loop, divides by len(numbers) to get the average
Prints a clear, formatted message

Output 17.5 is exactly right ☑

Now you’ve successfully:

Used for + range(len(...)) to loop by index
Used for value in list to loop by value

These are the two big patterns for list iteration in Python 🎯

I want to do one last quick check to lock this in, then we can wrap this lesson. Ready for a short multiple-choice question?

Yes, I'm ready

What do you think is the better loop here? Pick an option below 👇

You have this list:

```
prices = [4.99, 2.50, 10.00]
```

You want to print each price, and you don’t care about the index/position. Which loop is the better choice?

A.

```
for i in range(len(prices)):
    print(prices[i])
```

B.

```
for price in prices:
    print(price)
```

Exactly the right choice, Ron 🎯

`for price in prices:` is better here because:

You only care about the values, not their positions
Python hands you each price directly (4.99, then 2.50, then 10.00)
The code is shorter, cleaner, and easier to read:
Python

```
for price in prices:
    print(price)
```

No need for range or indexes when the index isn’t used ☑

You’ve now shown you can:

Loop with indices using `range(len(...))`
Loop directly over values with `for value in list`
Choose the right pattern based on whether you need positions or not

You’ve mastered this lesson’s goals 🎉

Ready to wrap up this lesson and move on to the next one?

Yes, let's move on

Nice work, Ron — that was a clean mastery of list iteration 🎉

Quick recap of what you can now do:

Use index-based loops:

```
for i in range(len(my_list)):
    print(i, my_list[i])
```

when you need the position (index) and the value.

Use value-based loops:

```
for item in my_list:
    print(item)
```

when you only care about the values and want simpler code.

You also practiced a real task (average of numbers) and chose the right pattern based on what you needed. That’s exactly how real-world Python is written 💪

You’re all set for the next lesson.
Whenever you’re ready, continue in the Learn tab and we’ll keep building on this.

How would you rate this lesson?